

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.:T001	Name:Soham Acharekar
Class:TYIT	Batch:1
Date of Assignment:28/7/26	Date/Time of Submission:28/7/26

Part A - Theory:

Part B - Program

a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Step 1: Start the application and open the first webpage.

Step 2: Create a cookie object using `HttpCookie`.

Step 3: Store user data (like name or preferences) in the cookie.

Step 4: Set cookie expiry time (session or persistent).

Step 5: Add the cookie to the browser using `Response.Cookies.Add()`.

Step 6: Navigate to another page.

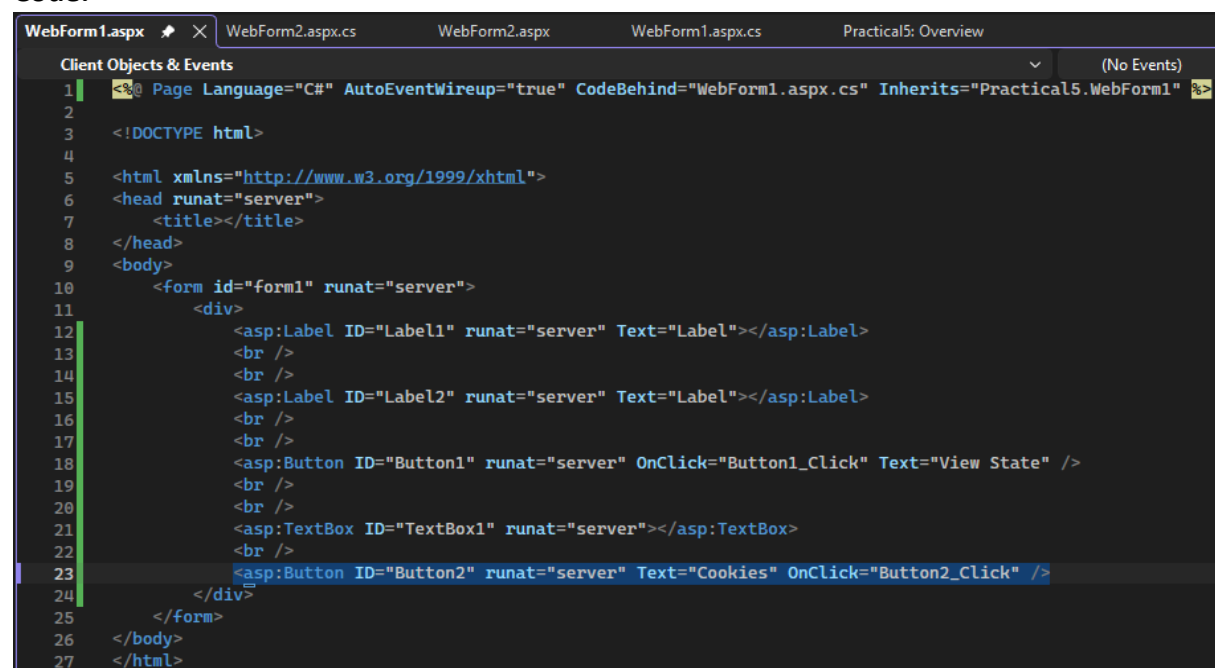
Step 7: Retrieve cookie using `Request.Cookies`.

Step 8: Display stored values on the second page.

Step 9: Handle null condition if cookie does not exist.

Step 10: End the program.

Code:



```

WebForm1.aspx WebForm2.aspx.cs WebForm2.aspx WebForm1.aspx.cs Practical5: Overview
Client Objects & Events (No Events)
1 <%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm1.aspx.cs" Inherits="Practical5.WebForm1" %>
2
3 <!DOCTYPE html>
4
5 <html xmlns="http://www.w3.org/1999/xhtml">
6 <head runat="server">
7 <title></title>
8 </head>
9 <body>
10 <form id="form1" runat="server">
11 <div>
12 <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
13 <br />
14 <br />
15 <asp:Label ID="Label2" runat="server" Text="Label"></asp:Label>
16 <br />
17 <br />
18 <asp:Button ID="Button1" runat="server" OnClick="Button1_Click" Text="View State" />
19 <br />
20 <br />
21 <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
22 <br />
23 <asp:Button ID="Button2" runat="server" Text="Cookies" OnClick="Button2_Click" />
24 </div>
25 </form>
26 </body>
27 </html>

```

```

WebForm1.aspx WebForm2.aspx.cs WebForm2.aspx WebForm1.aspx.cs Practical5: Overview
Client Objects & Events (No Events)
1 <%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm2.aspx.cs" Inherits="Practical5.WebForm2" %>
2
3 <!DOCTYPE html>
4
5 <html xmlns="http://www.w3.org/1999/xhtml">
6 <head runat="server">
7 <title></title>
8 </head>
9 <body>
10 <form id="form1" runat="server">
11 <div>
12 <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
13 </div>
14 </form>
15 </body>
16 </html>

```

```

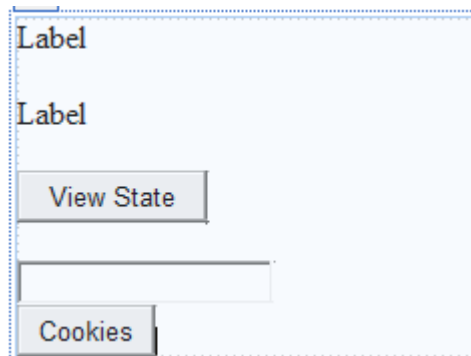
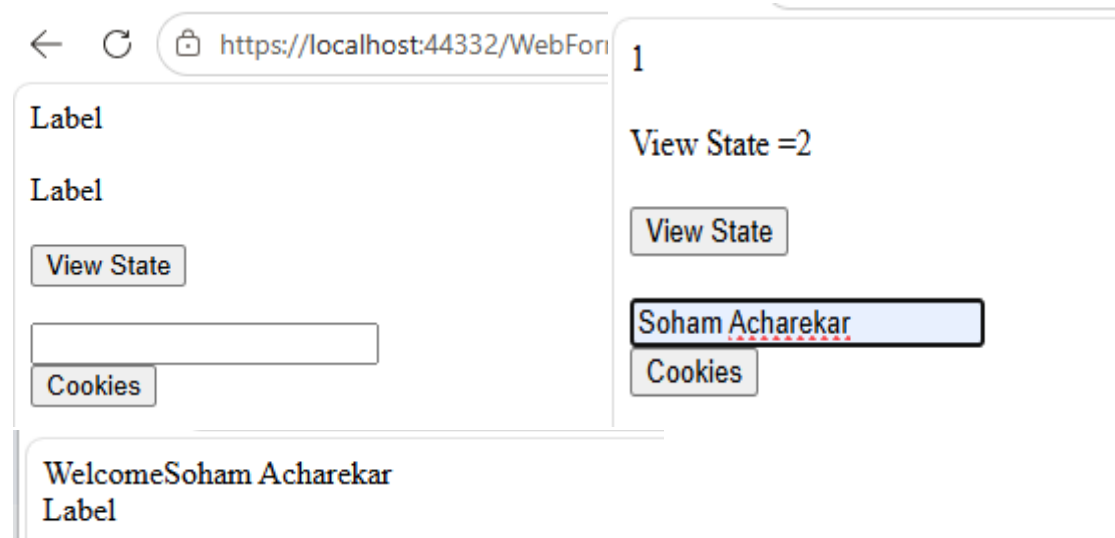
WebForm1.aspx WebForm2.aspx.cs WebForm2.aspx WebForm1.aspx.cs Practical5: Overview
Practical5 Practical5.WebForm1
8
9 namespace Practical5
10 {
11     2 references
12     public partial class WebForm1 : System.Web.UI.Page
13     {
14         0 references
15         protected void Page_Load(object sender, EventArgs e)
16         {
17             if(IsPostBack)
18             {
19                 namespace Practical5 : ViewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
20                 Label2.Text = "View State "+ViewStateVal.ToString();
21             }
22             else
23             {
24                 ViewState["count"] = 1;
25             }
26         }
27     }
28
29     0 references
30     protected void Button1_Click(object sender, EventArgs e)
31     {
32         Label1.Text = ViewState["count"].ToString();
33     }
34
35     0 references
36     protected void Button2_Click(object sender, EventArgs e)
37     {
38         HttpCookie h = new HttpCookie("name");
39         h.Value = TextBox1.Text;
40         Response.Cookies.Add(h);
41         Response.Redirect("WebForm2.aspx");
42     }
43 }

```

```

WebForm1.aspx WebForm2.aspx.cs WebForm2.aspx WebForm1.aspx.cs Practical5: Overview
Practical5 Practical5.WebForm2
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.UI;
6 using System.Web.UI.WebControls;
7
8 namespace Practical5
9 {
10     2 references
11     public partial class WebForm2 : System.Web.UI.Page
12     {
13         0 references
14         protected void Page_Load(object sender, EventArgs e)
15         {
16             if (Request.Cookies["name"]!=null)
17             {
18                 Response.Write("Welcome" + Request.Cookies["name"].Value);
19             }
20         }
21     }
22 }

```

DESIGN:**Output:**

b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

- Step 1:** Create [ASP.NET](#) Core project.
- Step 2:** Install EF Core packages.
- Step 3:** Create Product model class (Id, Name, Price).
- Step 4:** Create DbContext class and configure database connection.
- Step 5:** Run migration to create Product table in SQL Server.
- Step 6:** Create a form to accept product details.
- Step 7:** Capture user input in controller.
- Step 8:** Add product object to database using Add () method.
- Step 9:** Save changes using SaveChanges () .
- Step 10:** Display success message or inserted data.

Code:

The image shows a Visual Studio code editor with three files open: `HomeController.cs`, `Program.cs`, and `appsettings.json`. The `HomeController.cs` file contains a `HomeController` class that inherits from `Controller`. It has a private readonly `AppDbContext` property `_context` and a constructor that takes an `AppDbContext` parameter. The `Index` method creates a new `Product` object with `Name = "Laptop"` and `Price = 50000`, adds it to the `_context.Products` collection, saves the changes, and returns a content result.

The `Program.cs` file shows the application's startup logic. It uses `WebApplication.CreateBuilder` to create the builder, registers the MVC and EF Core services, adds the `AppDbContext` with the `UseSqlServer` provider, builds the application, and maps the controller route for the `Index` action.

The `appsettings.json` file contains the application's configuration, including logging settings, allowed hosts, and connection strings.

```

using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    private readonly AppDbContext _context;

    public HomeController(AppDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        Product p = new Product();
        {
            p.Name = "Laptop";
            p.Price = 50000;
        };

        _context.Products.Add(p);
        _context.SaveChanges();

        return Content("Product Inserted Successfully");
    }
}

```

```

using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Register MVC and EF Core
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}

```

```

SQLQuery1.sql *   HomeController.cs   Program.cs   appsettings.json   AppDbContext.cs   Product.cs
Practical5b
1  using Microsoft.EntityFrameworkCore;
2
3  7 references
4  public class AppDbContext : DbContext
5  {
6      0 references
7      public AppDbContext(DbContextOptions<AppDbContext> options)
8          : base(options)
9      {
10
11      1 reference
12  public DbSet<Product> Products { get; set; }

```

```

SQLQuery1.sql *   HomeController.cs   Program.cs   appsettings.json   AppDbContext.cs   Product.cs
Practical5b
1  3 references
2  public class Product
3  {
4      0 references
5      public int Id { get; set; }
6      1 reference
7      public string Name { get; set; }
8      1 reference
9      public int Price { get; set; }
10 }

```

Output:

Product Inserted Successfully

SQL Server Object Explorer

- SQL Server
 - (localdb)\MSSQLLocalDB (SQL Server 17.0.4025 - DE
 - Databases
 - System Databases
 - ProductDB
 - Tables
 - Views
 - Synonyms
 - Programmability
 - External Resources
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - (localdb)\ProjectModels (SQL Server 17.0.4025 - DES
 - Projects - Practicalb

SQLQuery1.sql * 2026072803115...itialCreate.cs

```

1  select * from products;
2

```

100 % No issues found

Id	Name	Price
1	T-SQL	50000

Query executed successfully at 8:47:50 AM

c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Algorithm:

Step 1: Create [ASP.NET](#) Core MVC project.

Step 2: Define Employee model (Id, Name, Salary, Department).

Step 3: Configure DbContext and connect to SQL Server.

Step 4: Create database using migrations.

Step 5: Implement Create operation (Add new employee).

Step 6: Implement Read operation (Display employee list).

Step 7: Implement Update operation (Edit employee details).

Step 8: Implement Delete operation (Remove employee record).

Step 9: Use EF Core methods (Add, Find, Update, Remove).

Step 10: Display results in views.

Code:

The screenshot displays the Visual Studio IDE with the EmployeeCRUD project open. The top pane shows the EmployeeCRUD.Models namespace with the Employee class. The bottom pane shows the AppDbContext class in EmployeeCRUD.Data.AppDbContext.

```

namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int ID { get; set; }

        public required string Name { get; set; }

        public required string Department { get; set; }

        public double Salary { get; set; }
    }
}

```

```

<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.10">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.10" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.10">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.VisualStudio.Web.CodeGeneration.Design" Version="10.0.2" />
  </ItemGroup>
  <ItemGroup>
    <Folder Include="Views\Employee\" />
  </ItemGroup>
</Project>

```

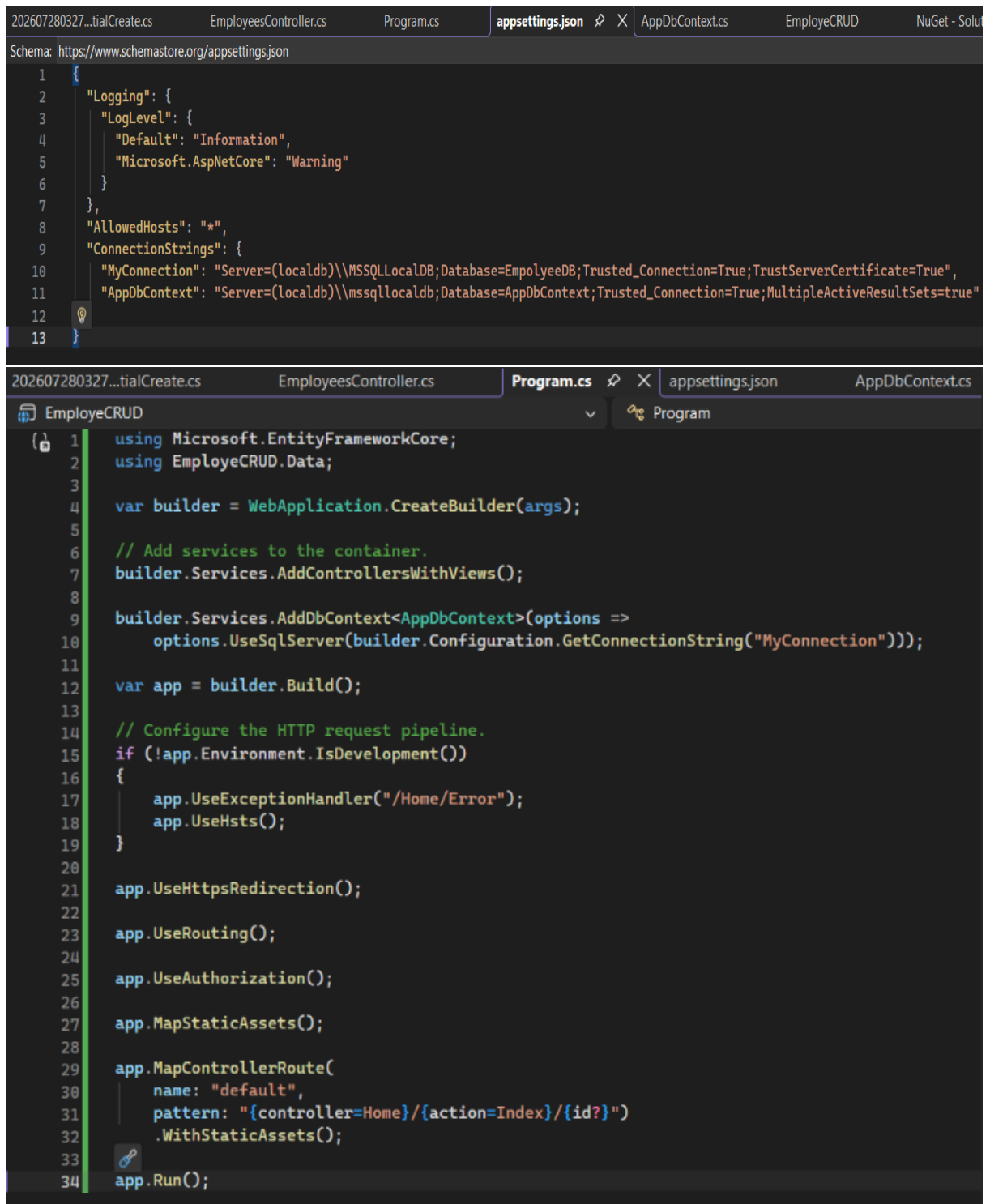
```

using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}

```



The screenshot displays two files in a Visual Studio editor. The top file is `appsettings.json`, which contains configuration for logging, allowed hosts, and connection strings. The bottom file is `Program.cs`, which shows the application's startup logic, including service registration and middleware configuration.

```
202607280327...tialCreate.cs EmployeesController.cs Program.cs appsettings.json AppDbContext.cs EmployeeCRUD NuGet - Solu
Schema: https://www.schemastore.org/appsettings.json
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*",
9   "ConnectionStrings": {
10    "MyConnection": "Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
11    "AppDbContext": "Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
12  }
13 }
```

```
202607280327...tialCreate.cs EmployeesController.cs Program.cs appsettings.json AppDbContext.cs
EmployeeCRUD
1 using Microsoft.EntityFrameworkCore;
2 using EmployeeCRUD.Data;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 // Add services to the container.
7 builder.Services.AddControllersWithViews();
8
9 builder.Services.AddDbContext<AppDbContext>(options =>
10     options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
11
12 var app = builder.Build();
13
14 // Configure the HTTP request pipeline.
15 if (!app.Environment.IsDevelopment())
16 {
17     app.UseExceptionHandler("/Home/Error");
18     app.UseHsts();
19 }
20
21 app.UseHttpsRedirection();
22
23 app.UseRouting();
24
25 app.UseAuthorization();
26
27 app.MapStaticAssets();
28
29 app.MapControllerRoute(
30     name: "default",
31     pattern: "{controller=Home}/{action=Index}/{id?}"
32     .WithStaticAssets());
33
34 app.Run();
```

```

20260728032...ialCreate.cs EmployeesController.cs Program.cs appsettings.json AppDb
EmployeeCRUD
1 using Microsoft.EntityFrameworkCore.Migrations;
2
3 #nullable disable
4
5 namespace EmployeeCRUD.Migrations
6 {
7     /// <inheritdoc />
8     1 reference
9     public partial class InitialCreate : Migration
10     {
11         /// <inheritdoc />
12         0 references
13         protected override void Up(MigrationBuilder migrationBuilder)
14         {
15             migrationBuilder.CreateTable(
16                 name: "Employees",
17                 columns: table => new
18                 {
19                     ID = table.Column<int>(type: "int", nullable: false)
20                         .Annotation("SqlServer:Identity", "1, 1"),
21                     Name = table.Column<string>(type: "nvarchar(max)", nullable: false),
22                     Department = table.Column<string>(type: "nvarchar(max)", nullable: false),
23                     Salary = table.Column<double>(type: "float", nullable: false)
24                 },
25                 constraints: table =>
26                 {
27                     table.PrimaryKey("PK_Employees", x => x.ID);
28                 });
29         }
30
31         /// <inheritdoc />
32         0 references
33         protected override void Down(MigrationBuilder migrationBuilder)
34         {
35             migrationBuilder.DropTable(
36                 name: "Employees");
37         }
38     }
39 }

```

Output:

Index

[Create New](#)

Name	Department	Salary	
Pranay	Development	80000	Edit Details Delete
Soham	Web Developer	60000	Edit Details Delete
Siddhesh	Developer	75000	Edit Details Delete

d) Authentication and Authorization in ASP.NET Core.

Algorithm:

- Step 1: Create [ASP.NET](#) Core project with Identity.
- Step 2: Configure authentication services in Program.cs.
- Step 3: Add login and registration functionality.
- Step 4: Store user credentials securely in database.
- Step 5: Enable authentication middleware.
- Step 6: Use [Authorize] attribute to restrict access.
- Step 7: Define roles (Admin, User).
- Step 8: Assign roles to users.
- Step 9: Allow access based on roles.
- Step 10: Redirect unauthorized users to login page.

Code:

The image shows a Visual Studio code editor with two files open. The top file is Program.cs, which contains the startup code for an ASP.NET Core application. The bottom file is Index.cshtml, which contains the HTML for a login form.

```

Index.cshtml  HomeController.cs*  Program.cs  Practical5d: Overview
Practical5d
1  using Microsoft.AspNetCore.Authentication.Cookies;
2
3  var builder = WebApplication.CreateBuilder(args);
4
5  builder.Services.AddControllersWithViews();
6
7  builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
8  .AddCookie(options =>
9  {
10     options.LoginPath = "/Home/Index";
11 });
12
13 builder.Services.AddAuthorization();
14
15 var app = builder.Build();
16
17 app.UseStaticFiles();
18
19 app.UseRouting();
20
21 app.UseAuthentication();
22 app.UseAuthorization();
23
24 app.MapControllerRoute(
25     name: "default",
26     pattern: "{controller=Home}/{action=Index}/{id?}");
27
28 app.Run();
Index.cshtml  HomeController.cs*  Program.cs  Practical5d: Overview
1  <h2>Login</h2>
2
3  <form method="post">
4
5      Username :
6      <input type="text" name="username" /><br /><br />
7
8      Password :
9      <input type="password" name="password" /><br /><br />
10
11     <input type="submit" value="Login" />
12
13 </form>
14
15 <p style="color: red">
16     @ViewBag.Message
17 </p>
  
```

```
Index.cshtml | HomeController.cs* | Program.cs | Practical5d: Ov
Practical5d
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.AspNetCore.Authorization;
3 using Microsoft.AspNetCore.Authentication;
4 using Microsoft.AspNetCore.Authentication.Cookies;
5 using System.Security.Claims;
6 namespace AuthDemo.Controllers
7 {
8     0 references
9     public class HomeController : Controller
10    {
11        0 references
12        public IActionResult Index()
13        {
14            return View();
15        }
16        [HttpPost]
17        0 references
18        public async Task<IActionResult> Index(string username, string password)
19        {
20            if (username == "admin" && password == "123")
21            {
22                var claims = new List<Claim>
23                {
24                    new Claim(ClaimTypes.Name, username)
25                };
26                var identity = new ClaimsIdentity(claims,
27                    CookieAuthenticationDefaults.AuthenticationScheme);
28                await HttpContext.SignInAsync(
29                    CookieAuthenticationDefaults.AuthenticationScheme,
30                    new ClaimsPrincipal(identity));
31                return RedirectToAction("Secure");
32            }
33            ViewBag.Message = "Invalid Username or Password";
34            return View();
35        }
36        [Authorize]
37        0 references
38        public IActionResult Secure()
39        {
40            return Content("Welcome Soham Acharekar!Login Successful.");
41        }
42        0 references
43        public async Task<IActionResult> Logout()
44        {
45            await HttpContext.SignOutAsync(
46                CookieAuthenticationDefaults.AuthenticationScheme);
47            return RedirectToAction("Index");
48        }
49    }
50 }
```

Output:

Login

Username :

Password :

Login

Welcome Soham Acharekar!Login Successful.